# REST API Rekening Bank & Transaksi

## Objective

- Mengimplementasikan operasi CRUD pada data rekening bank.
- Merancang database dengan model relasional yang mencakup minimal entitas untuk bank, rekening, dan transaksi.
- Menyediakan logika validasi dan error handling.

Specification

1. Model :

   - Bank
   - Accounts
   - Transactions

2. Fitur API:

   - Create Rekening: Endpoint: POST **/accounts** Body:
     Req:

```
{
    "account_number" : "1234567890",
    "account_holder" : "thomas",
    "balance" : 500000
}
```

     Res:

```
{
    "message": "Account created successfully",
    "data": {
        "id": "uuid1",
        "account_number": "1234567890",
        "account_holder": "John Doe",
        "balance": 500000,
        "created_at": "2025-01-08T15:04:05Z"
    }
}
```

- Get All Rekening: Endpoint: GET **/accounts**

  Res:

```json
[
    {
        "id": "uuid1",
        "account_number": "1234567890",
        "account_holder": "John Doe",
        "balance": 500000,
        "created_at": "2025-01-08T15:04:05Z",
        "updated_at": "2025-01-08T15:04:05Z"
    },
    {
        "id": "uuid2",
        "account_number": "9876543210",
        "account_holder": "Jane Smith",
        "balance": 1000000,
        "created_at": "2025-01-08T15:04:05Z",
        "updated_at": "2025-01-08T15:04:05Z"
    }
]
```

- Get Rekening by id: Endpoint: GET **/accounts/:id**
  Res:

```json
{
    "id": "uuid1",
    "account_number": "1234567890",
    "account_holder": "John Doe",
    "balance": 500000,
    "created_at": "2025-01-08T15:04:05Z",
    "updated_at": "2025-01-08T15:04:05Z"
}
```

- Update Rekening: Endpoint: PUT **/accounts/:id**

  Req:

```
{
    "account_holder" : "thomas",
    "balance" : 700000
}
```

  Res:

```
{
    "message": "Account updated successfully",
    "data": {
        "id": "uuid1",
        "account_number": "1234567890",
        "account_holder": "John Doe Updated",
        "balance": 700000,
        "created_at": "2025-01-08T15:04:05Z",
        "updated_at": "2025-01-08T15:04:05Z"
    }
}
```

- Delete Rekening: Endpoint: DELETE **/accounts/:id**

  Res:

```
{
   "message" : "Account deleted successfully"
}
```

- Transfer Uang Antar Rekening: Endpoint: POST **/transfer**

  Req:

```
{
  "from_account_id": "UUID-account-asal",
  "to_account_id": "UUID-account-tujuan",
  "amount": 100000
}
```

  Res:

```
Response: Detail transaksi transfer beserta update saldo rekening asal dan
tujuan. Error Handling: - Error jika salah satu ID rekening tidak ditemukan. -
Error jika saldo rekening asal tidak mencukupi untuk melakukan transfer.
```

- Histori Transaksi Rekening:

  Endpoint: GET /accounts/:id/transactions

  Response: Daftar seluruh transaksi yang terkait dengan rekening tersebut.

  Error Handling: Error jika rekening dengan ID tersebut tidak ditemukan.

3. Error Handling untuk setiap fitur endpoint List error handling :

- error create saat input account_number yang sama
- error saat create body json kosong
- error saat membaca data id yang tidak ada
- error saat mengupdate data id yang tidak ada
- error saat menghapus data id yang tidak ada

```
go get github.com/lib/pq go get github.com/google/uuid
```